

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

Программирование на языке Ассемблера
по дисциплине «Архитектура компьютера»

Выполнил

студент гр. 5130904/40003

<

>

А.Д.Николаев

Руководитель

проф, д.т.н.

<

>

С.А.Молодяков

«17» ноября 2025г.

Санкт-Петербург

2025

Оглавление

Оглавление	2
Введение	3
Задание.....	4
Задача 1	4
Задача 2	4
Задача 1	5
Блок-схемы и алгоритмы выполнения программы	5
Системные прерывания (их аналоги)	7
Листинг программы с комментариями с макросами.....	8
Листинг (скриншоты) результатов выполнения задания.....	11
Задача 2	12
Блок-схемы	12
Системные прерывания (их аналоги)	14
Листинг программы с комментариями с макросами.....	15
Листинг (скриншоты) результатов выполнения задания.....	19
Заключение.....	20
Список литературы.....	21

Введение

RISC-V – современная и перспективная архитектура, в отличие от других является более открытой (команды и спецификация RISC-V доступны абсолютно бесплатно, в отличие от ARM, x86 и др.), что делает её отличным выбором для изучения программирования на языке ассемблер. Основная идея RISC-архитектуры (с сокращенным набором команд) заключается в том, что процессор работает наиболее эффективно во время выполнения небольшого набора команд и инструкций.

Для выполнения работы был выбран симулятор RARS (что расшифровывается как RISC-V Assembler and Runtime Simulator), который разработал профессор из Университета Миссури, США. Это удобная программа, которая позволяет запускать код для RISC-V на любом поддерживаемом устройстве.

Задания

Задача 1

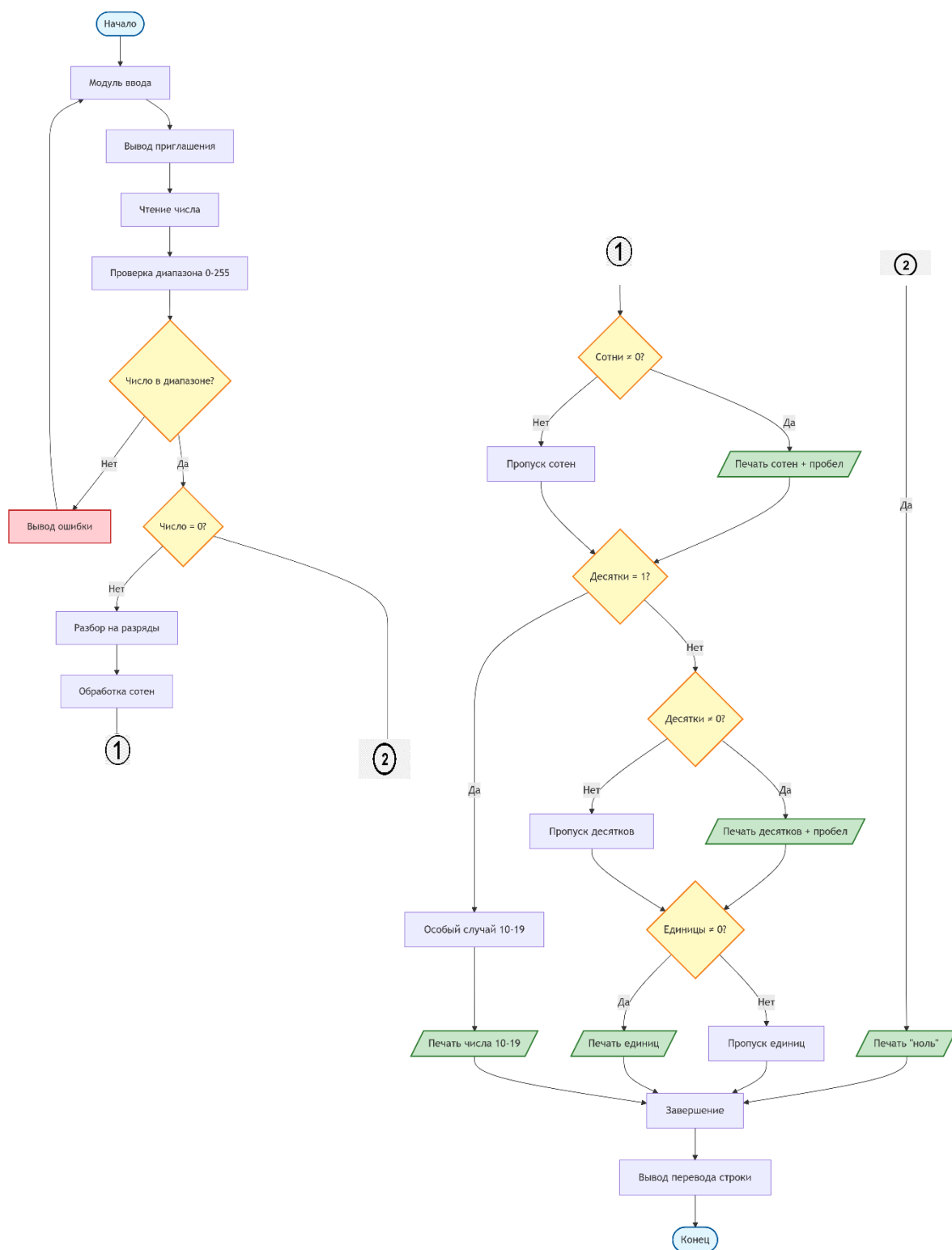
Написать программу, записывающую однобайтовое целое число без знака прописью, например, 128 = сто двадцать восемь

Задача 2

Мультизадачность 2. Организуйте мультизадачную среду (3 задачи) с выводом на экран номера задачи. Переключение происходит по алгоритму кругового планирования с выделенными квантами времени.

Задача 1

Блок-схемы и алгоритмы выполнения программы



Общий алгоритм программы:

1. Ввод числа пользователем.
2. Проверка диапазона (0–255). При ошибке — повтор ввода.
3. Разложение числа на сотни, десятки и единицы.
4. Вызов соответствующих макросов вывода (HundredsModule, TensModule, UnitsModule).
5. Завершение программы (EndModule).

Также используются следующие макросы:

- **InputModule** - Вводит число, проверяет корректность. При неверном вводе — сообщение об ошибке и возврат к началу.
- **HundredsModule** - Если число ≥ 100 , выбирает и выводит слово из таблицы сотен («сто», «двести» и т.д.).
- **TensModule** - Выводит десятки. При значении 1 вызывает SpecialModule (числа 10–19).
- **UnitsModule** - Выводит единицы, если они есть, выбирая слово из таблицы.
- **SpecialModule** - Обработывает числа 10–19, выводя соответствующее слово.
- **EndModule** - Печатает перевод строки и завершает выполнение программы.

Системные прерывания (их аналоги)

В архитектуре RISC-V нет системных прерываний в понимании системы DOS – тут используется их аналог – системный вызов – они и используются в решении задачи.

RISC-V	Действие	Аналог в DOS
$a7 = 4$	Печать строки (print_string) Используется для вывода текстов: приглашения к вводу, чисел словами, пробела, перевода строки и сообщений об ошибках.	INT 21h, AH = 09h
$a7 = 5$	Ввод целого числа (read_int) Считывает число, введенное пользователем с клавиатуры, и сохраняет его в $a0$.	INT 21h, AH = 01h
$a7 = 10$	Завершение программы (exit) Останавливает выполнение и возвращает управление операционной системе.	INT 21h, AH = 4Ch

Листинг программы с комментариями с макросами

```
1 .data
2 # Массивы указателей на строки с текстовыми представлениями чисел
3 units: .word unit0, unit1, unit2, unit3, unit4, unit5, unit6, unit7, unit8, unit9
4 tens: .word ten0, ten1, ten2, ten3, ten4, ten5, ten6, ten7, ten8, ten9
5 hundreds: .word hundred0, hundred1, hundred2
6
7 # Текстовые представления единиц (0-9)
8 unit0: .asciz "" # пустая строка для 0
9 unit1: .asciz "один"
10 unit2: .asciz "два"
11 unit3: .asciz "три"
12 unit4: .asciz "четыре"
13 unit5: .asciz "пять"
14 unit6: .asciz "шесть"
15 unit7: .asciz "семь"
16 unit8: .asciz "восемь"
17 unit9: .asciz "девять"
18
19 # Текстовые представления десятков (0-90)
20 ten0: .asciz "" # пустая строка для 0
21 ten1: .asciz "десять"
22 ten2: .asciz "двадцать"
23 ten3: .asciz "тридцать"
24 ten4: .asciz "сорок"
25 ten5: .asciz "пятьдесят"
26 ten6: .asciz "шестьдесят"
27 ten7: .asciz "семьдесят"
28 ten8: .asciz "восемьдесят"
29 ten9: .asciz "девяносто"
30
31 # Текстовые представления сотен (0-200)
32 hundred0: .asciz "" # пустая строка для 0
33 hundred1: .asciz "сто"
34 hundred2: .asciz "двести"
35
36 # Особые случаи: числа 10-19
37 special10: .asciz "десять"
38 special11: .asciz "одиннадцать"
39 special12: .asciz "двенадцать"
40 special13: .asciz "тринадцать"
41 special14: .asciz "четырнадцать"
42 special15: .asciz "пятнадцать"
43 special16: .asciz "шестнадцать"
44 special17: .asciz "семнадцать"
45 special18: .asciz "восемнадцать"
46 special19: .asciz "девятнадцать"
47
48 # Массив указателей на особые случаи (10-19)
49 special: .word special10, special11, special12, special13, special14, special15, special16,
special17, special18, special19
50
51 # Константы и строки для ввода/вывода
52 max_value: .word 255 # максимальное допустимое значение
53 space: .asciz " " # строка пробела для разделения слов
54 newline: .asciz "\n" # строка перевода строки
55 prompt: .asciz "Введите число от 0 до 255: " # приглашение для ввода
56 zero_text: .asciz "ноль" # текстовое представление нуля
57 error_msg: .asciz "Ошибка: число должно быть от 0 до 255!\n" # сообщение об ошибке
58
59 .text
60
61 # -----
62 # Определения макросов
63 # -----
64
65 # InputModule: вывести prompt, прочитать int -> s0, проверить диапазон
66 .macro InputModule
67     la a0, prompt # загрузить адрес строки приглашения в a0
68     li a7, 4 # системный вызов для печати строки
69     ecall # выполнить системный вызов
70
71     li a7, 5 # системный вызов для чтения целого числа
72     ecall # выполнить системный вызов
73     mv s0, a0 # сохранить введенное число в s0
74
75     blt s0, x0, input_error # если число < 0, перейти к ошибке
76     lw t0, max_value # загрузить максимальное значение (255) в t0
```



```

77     bgt s0, t0, input_error    # если число > 255, перейти к ошибке
78 .end_macro
79
80 # HundredsModule: печать сотен (s1), затем пробел (если s1 != 0)
81 .macro HundredsModule
82     beqz s1, .HundredsDone      # если сотен нет (s1 = 0), пропустить печать
83     la t0, hundreds            # загрузить адрес массива сотен
84     slli t1, s1, 2              # умножить номер сотен на 4 (размер слова)
85     add t0, t0, t1              # вычислить адрес элемента массива
86     lw a0, 0(t0)               # загрузить адрес строки с текстом сотен
87     li a7, 4                   # системный вызов для печати строки
88     ecall                      # выполнить системный вызов
89
90     la a0, space                # загрузить адрес строки с пробелом
91     li a7, 4                   # системный вызов для печати строки
92     ecall                      # выполнить системный вызов
93 .HundredsDone:
94 .end_macro
95
96 # TensModule: если s3==1 -> handle_special_case (10..19),
97 # иначе печать десятков (если s3 != 0) и пробел
98 .macro TensModule
99     li t0, 1                   # загрузить значение 1 для сравнения
100    beq s3, t0, handle_special_case # если десятки = 1, обработать особый случай (10-19)
101    beqz s3, .TensDone          # если десяток нет (s3 = 0), пропустить печать
102
103    la t0, tens                 # загрузить адрес массива десятков
104    slli t1, s3, 2              # умножить номер десятков на 4 (размер слова)
105    add t0, t0, t1              # вычислить адрес элемента массива
106    lw a0, 0(t0)               # загрузить адрес строки с текстом десятков
107    li a7, 4                   # системный вызов для печати строки
108    ecall                      # выполнить системный вызов
109
110    la a0, space                # загрузить адрес строки с пробелом
111    li a7, 4                   # системный вызов для печати строки
112    ecall                      # выполнить системный вызов
113 .TensDone:
114 .end_macro
115
116 # UnitsModule: печать единиц (s4) если s4 != 0
117 .macro UnitsModule
118     beqz s4, .UnitsDone        # если единиц нет (s4 = 0), пропустить печать
119     la t0, units               # загрузить адрес массива единиц
120     slli t1, s4, 2              # умножить номер единиц на 4 (размер слова)
121     add t0, t0, t1              # вычислить адрес элемента массива
122     lw a0, 0(t0)               # загрузить адрес строки с текстом единиц
123     li a7, 4                   # системный вызов для печати строки
124     ecall                      # выполнить системный вызов
125 .UnitsDone:
126 .end_macro
127
128 # SpecialModule: печать 10..19 по таблице special (индексируется s4)
129 .macro SpecialModule
130     la t0, special              # загрузить адрес массива особых случаев
131     slli t1, s4, 2              # умножить номер на 4 (размер слова)
132     add t0, t0, t1              # вычислить адрес элемента массива
133     lw a0, 0(t0)               # загрузить адрес строки с текстом числа 10-19
134     li a7, 4                   # системный вызов для печати строки
135     ecall                      # выполнить системный вызов
136 .end_macro
137
138 # EndModule: перевод строки + exit
139 .macro EndModule
140     la a0, newline              # загрузить адрес строки с переводом строки
141     li a7, 4                   # системный вызов для печати строки
142     ecall                      # выполнить системный вызов
143
144     li a7, 10                  # системный вызов для завершения программы
145     ecall                      # выполнить системный вызов
146 .end_macro
147
148 # -----
149 # Основная программа
150 # -----
151 main:
152     InputModule                 # ввод и проверка (записывает s0)
153
154     beqz s0, print_zero         # если введен 0 -> печатаем "ноль" и завершаем
155
156     # Разбиение числа на разряды

```

```

157     li t0, 100                # загрузить делитель 100
158     div s1, s0, t0            # s1 = сотни (s0 / 100)
159     rem s2, s0, t0            # s2 = остаток после сотен (s0 % 100)
160
161     li t0, 10                 # загрузить делитель 10
162     div s3, s2, t0            # s3 = десятки (s2 / 10)
163     rem s4, s2, t0            # s4 = единицы (s2 % 10)
164
165     # Последовательная обработка разрядов
166     HundredsModule             # обработка сотен
167     # продолжение — обработка десятков/единиц
168     TensModule                 # обработка десятков
169     UnitsModule                # обработка единиц
170     j end_program              # переход к завершению программы
171
172 # Обработка особых случаев (числа 10-19)
173 handle_special_case:
174     SpecialModule              # печать числа 10-19
175     j end_program              # переход к завершению программы
176
177 # Печать нуля
178 print_zero:
179     la a0, zero_text           # загрузить адрес строки "ноль"
180     li a7, 4                   # системный вызов для печати строки
181     ecall                     # выполнить системный вызов
182     j end_program              # переход к завершению программы
183
184 # Обработка ошибки ввода
185 input_error:
186     la a0, error_msg           # загрузить адрес сообщения об ошибке
187     li a7, 4                   # системный вызов для печати строки
188     ecall                     # выполнить системный вызов
189     j main                     # вернуться к началу программы для повторного ввода
190
191 # Место завершения программы
192 end_program:
193     EndModule                  # завершение программы (перевод строки + exit)

```

Листинг (скриншоты) результатов выполнения задания

```
Введите число от 0 до 255: Введите число от 0 до 255: 123  
сто двадцать три
```

```
-- program is finished running (0) --
```

```
Введите число от 0 до 255: 0  
ноль
```

```
-- program is finished running (0) --
```

```
Введите число от 0 до 255: 12  
двенадцать
```

```
-- program is finished running (0) --
```

```
Введите число от 0 до 255: 111  
сто одиннадцать
```

```
-- program is finished running (0) --
```

```
Введите число от 0 до 255: 100  
сто
```

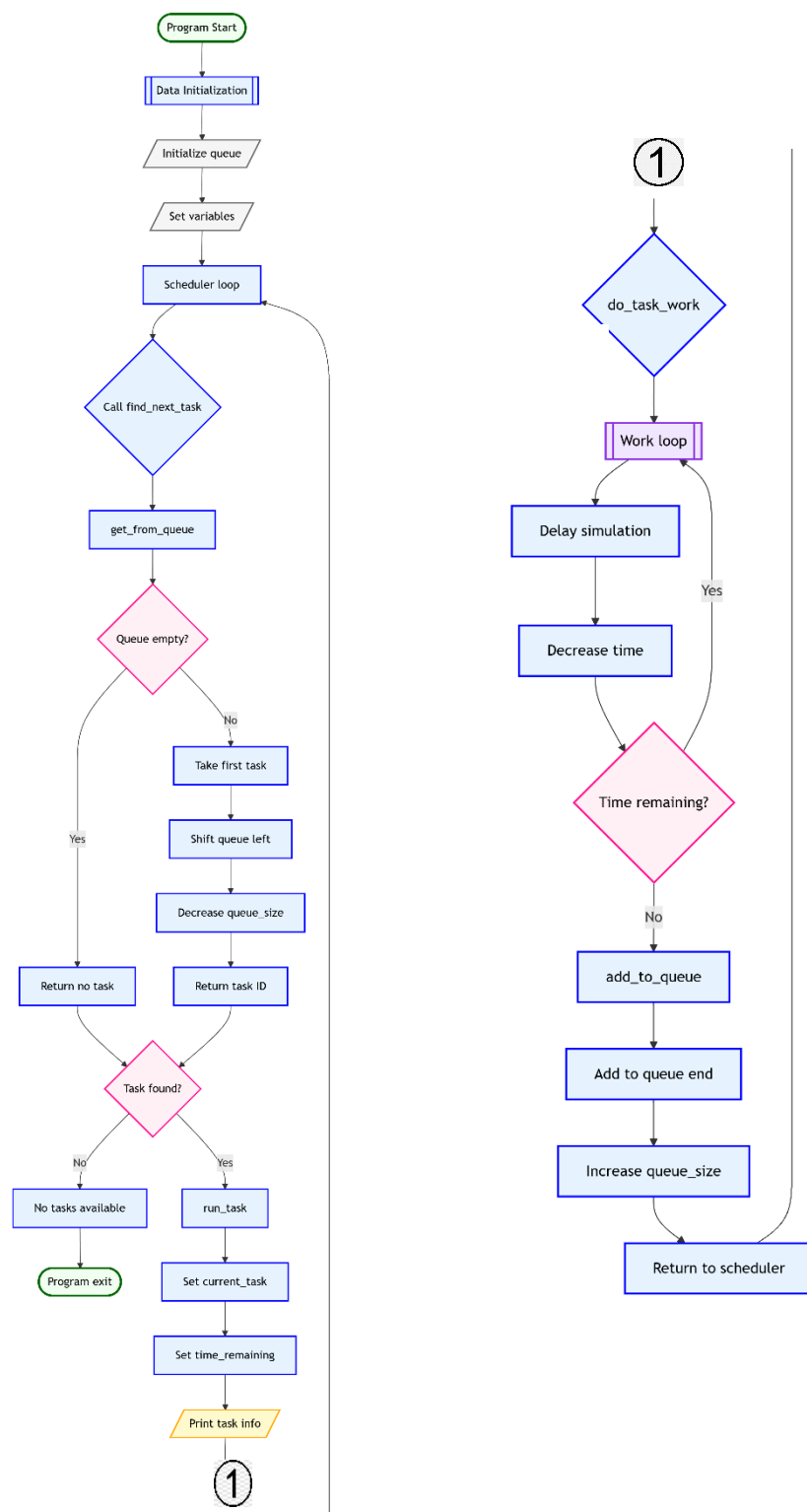
```
-- program is finished running (0) --
```

```
Введите число от 0 до 255: 128  
сто двадцать восемь
```

```
-- program is finished running (0) --
```

Задача 2

Блок-схемы



Общий алгоритм программы по пунктам:

1. Инициализация - заполняем очередь тремя задачами (0, 1, 2) и устанавливаем размер очереди = 3
2. Запуск планировщика - переходим в основной цикл планировщика
3. Поиск задачи - берем первую задачу из начала очереди
4. Проверка - если очередь пустая, завершаем программу
5. Выполнение задачи:
 - Запоминаем текущую задачу
 - Устанавливаем квант времени = 8
 - Выводим информацию о задаче
 - Выполняем работу (уменьшаем время до 0)
 - Возвращаем задачу в конец очереди
6. Повтор - возвращаемся к шагу 3

Также используются следующие макросы:

- `find_next_task` - Ищет следующую задачу для выполнения. Если очередь пуста, возвращает ошибку.
- `run_task` - Запускает выполнение задачи: устанавливает квант времени, выводит информацию о задаче и вызывает её выполнение.
- `do_task_work` - Выполняет работу задачи в течение выделенного кванта времени, уменьшая оставшееся время до нуля.
- `add_to_queue` - Добавляет задачу в конец очереди после завершения её кванта времени.
- `get_from_queue` - Извлекает задачу из начала очереди для выполнения. При пустой очереди возвращает -1.

Системные прерывания (их аналоги)

RISC-V	Действие	Аналог в DOS
a7 = 4	Печать строки (print_string) Используется для вывода текстов: приглашения к вводу, чисел словами, пробела, перевода строки и сообщений об ошибках.	INT 21h, AH = 09h
a7 = 1	Вывод одного символа/числа (print_symbol) Выводит один символ/число в консольку	INT 21h, AH = 02h
a7 = 10	Завершение программы (exit) Останавливает выполнение и возвращает управление операционной системе.	INT 21h, AH = 4Ch

Листинг программы с комментариями с макросами

```
1  .data
2  task_count: .word 3
3  quantum: .word 8
4
5  queue: .word -1, -1, -1
6  queue_size: .word 0
7
8  current_task: .word -1
9  time_remaining: .word 0
10
11 newline: .string "\n"
12 task_msg: .string "Task "
13 executing_msg: .string " executing (quantum: "
14 dot: .string ")\n"
15
16 .text
17 .globl main
18 main:
19     la t0, queue # t0 - указатель на очередь
20     li t1, 0 # Первая задача с ID 0
21     sw t1, 0(t0) # Кидаем в начало очереди
22     li t1, 1 # Вторая задача
23     sw t1, 4(t0) # Следующая ячейка
24     li t1, 2 # Третья задача
25     sw t1, 8(t0) # В конец
26
27     la t0, queue_size # Смотрим на размер очереди
28     li t1, 3 # У нас три задачи
29     sw t1, 0(t0) # Запоминаем размер
30
31     # Погнали планировщик
32     j scheduler
33
34 scheduler:
35     jal find_next_task # Ищем, какую задачу выполнять next
36     li t0, 1 # Проверяем, нашли ли что-то
37     bne a0, t0, no_tasks # Если нет задач - всё, конец
38
39     # Запускаем то, что нашли
40     mv a0, a1 # Перекидываем ID задачи
41     jal run_task # Выполняем её
42
43     j scheduler # И по новой
44
45 no_tasks:
46     li a7, 10 # Выходим из программы
47     ecall
48
49 # Функция поиска следующей задачи
50 # Возвращает: a0=1 если ок, a1=ID задачи
51 find_next_task:
52     addi sp, sp, -4 # Место в стеке
53     sw ra, 0(sp) # Сохраняем куда возвращаться
54
55     jal get_from_queue # Тянем задачу из очереди
56     li t0, -1 # Проверка на пустоту
57     bne a1, t0, found_task # Если не -1, значит нашли
58
59     # Не нашли ничего
60     li a0, 0 # Ставим флаг "не найдено"
61     li a1, -1 # ID = -1
62     j find_done # Выходим
63
64 found_task:
65     li a0, 1 # Флаг "найдено"
66     # В a1 уже лежит ID задачи
67
68 find_done:
69     lw ra, 0(sp) # Восстанавливаем возврат
70     addi sp, sp, 4 # Чистим стек
71     ret
72
73 # Запускаем задачу
74 # a0 = ID задачи
75 run_task:
76     addi sp, sp, -12 # Место для трёх регистров
77     sw ra, 0(sp) # Куда возвращаться
```

```

78     sw s0, 4(sp) # Сохраняем s0
79     sw s1, 8(sp) # И s1
80
81     mv s0, a0 # Запоминаем ID задачи в s0
82
83     # Запоминаем текущую задачу
84     la t0, current_task
85     sw s0, 0(t0) # Пишем что сейчас выполняем
86
87     # Ставим квант времени
88     la t0, quantum
89     lw t1, 0(t0) # Берем значение кванта
90     la t0, time_remaining
91     sw t1, 0(t0) # Записываем как оставшееся время
92
93     # Выводим инфу о задаче
94     la a0, task_msg # "Task "
95     li a7, 4
96     ecall
97
98     mv a0, s0 # Выводим ID задачи
99     li a7, 1
100    ecall
101
102    la a0, executing_msg # " executing (quantum: "
103    li a7, 4
104    ecall
105
106    la t0, time_remaining # Берем оставшееся время
107    lw a0, 0(t0)
108    li a7, 1
109    ecall
110
111    la a0, dot # ")\n"
112    li a7, 4
113    ecall
114
115    # Выполняем работу задачи
116    jal do_task_work # Делаем что-то полезное
117
118    # После выполнения кидаем задачу обратно в очередь
119    mv a0, s0 # ID задачи
120    jal add_to_queue # Добавляем в конец
121
122    lw ra, 0(sp) # Восстанавливаем всё
123    lw s0, 4(sp)
124    lw s1, 8(sp)
125    addi sp, sp, 12
126    ret
127
128    # Функция работы задачи (простая заглушка)
129    do_task_work:
130        addi sp, sp, -8 # Место для двух регистров
131        sw ra, 0(sp)
132        sw s0, 4(sp)
133
134        la s0, time_remaining # Указатель на оставшееся время
135        lw t0, 0(s0) # Текущее значение
136
137    work_loop:
138        # Простая задержка (типа работаем)
139        li t1, 0 # Счетчик
140        li t2, 300000 # До сколько считать
141
142    delay_loop:
143        addi t1, t1, 1 # +1 к счетчику
144        blt t1, t2, delay_loop # Пока не досчитали
145
146        # Уменьшаем оставшееся время
147        lw t0, 0(s0) # Перезагружаем время
148        addi t0, t0, -1 # Минус одна единица
149        sw t0, 0(s0) # Сохраняем
150
151        # Проверяем, не закончилось ли время
152        bgtz t0, work_loop # Если еще есть время - продолжаем
153
154        lw ra, 0(sp) # Восстанавливаем
155        lw s0, 4(sp)
156        addi sp, sp, 8
157        ret

```



```

158
159 # Добавляем задачу в конец очереди
160 # a0 = ID задачи
161 add_to_queue:
162     addi sp, sp, -12 # Место для трёх регистров
163     sw ra, 0(sp)
164     sw s0, 4(sp)
165     sw s1, 8(sp)
166
167     mv s0, a0 # Запоминаем ID задачи
168
169     la s1, queue # Указатель на очередь
170     la t0, queue_size # Указатель на размер
171
172     # Добавляем в конец
173     lw t1, 0(t0) # Текущий размер
174     slli t2, t1, 2 # Умножаем на 4 (смещение)
175     add t2, s1, t2 # Адрес куда пишем
176     sw s0, 0(t2) # Записываем задачу
177
178     # Увеличиваем размер
179     addi t1, t1, 1 # +1 к размеру
180     sw t1, 0(t0) # Сохраняем новый размер
181
182     lw ra, 0(sp) # Восстанавливаем
183     lw s0, 4(sp)
184     lw s1, 8(sp)
185     addi sp, sp, 12
186     ret
187
188 # Берем задачу из начала очереди
189 # Возвращает: a1 = ID задачи (-1 если пусто)
190 get_from_queue:
191     addi sp, sp, -12 # Место для трёх регистров
192     sw ra, 0(sp)
193     sw s0, 4(sp)
194     sw s1, 8(sp)
195
196     la s0, queue # Указатель на очередь
197     la s1, queue_size # Указатель на размер
198
199     lw t0, 0(s1) # Смотрим размер
200     beqz t0, queue_empty # Если ноль - очередь пуста
201
202     # Берем первую задачу
203     lw a1, 0(s0) # Первый элемент в a1
204
205     # Сдвигаем всю очередь влево
206     li t1, 1 # Начинаем со второго элемента
207
208 shift_loop:
209     bge t1, t0, shift_done # Если дошли до конца
210     slli t2, t1, 2 # Смещение для текущего
211     add t2, s0, t2 # Адрес текущего
212     lw t3, 0(t2) # Значение текущего
213     addi t4, t1, -1 # Индекс предыдущего
214     slli t4, t4, 2 # Смещение предыдущего
215     add t4, s0, t4 # Адрес предыдущего
216     sw t3, 0(t4) # Сдвигаем влево
217     addi t1, t1, 1 # Следующий элемент
218     j shift_loop # Продолжаем
219
220 shift_done:
221     # Обнуляем последний элемент
222     addi t0, t0, -1 # Новый размер (на 1 меньше)
223     slli t1, t0, 2 # Смещение последнего
224     add t1, s0, t1 # Адрес последнего
225     li t2, -1 # Значение -1
226     sw t2, 0(t1) # Обнуляем последний
227
228     # Уменьшаем размер
229     lw t0, 0(s1) # Текущий размер
230     addi t0, t0, -1 # -1
231     sw t0, 0(s1) # Сохраняем
232     j get_done
233
234 queue_empty:
235     li a1, -1 # Возвращаем -1 (ничего нет)
236
237 get_done:

```

```
238     lw ra, 0(sp) # Восстанавливаем
239     lw s0, 4(sp)
240     lw s1, 8(sp)
241     addi sp, sp, 12
242     ret
```

Листинг (скриншоты) результатов выполнения задания

```
Task 0 executing (quantum: 8)
Task 1 executing (quantum: 8)
Task 2 executing (quantum: 8)
Task 0 executing (quantum: 8)
Task 1 executing (quantum: 8)
Task 2 executing (quantum: 8)
Task 0 executing (quantum: 8)
```

Clear

```
Task 0 executing (quantum: 2)
Task 1 executing (quantum: 2)
Task 2 executing (quantum: 2)
Task 0 executing (quantum: 2)
Task 1 executing (quantum: 2)
Task 2 executing (quantum: 2)
Task 0 executing (quantum: 2)
Task 1 executing (quantum: 2)
Task 2 executing (quantum: 2)
Task 0 executing (quantum: 2)
Task 1 executing (quantum: 2)
```

Clear

Заключение

В ходе выполнения курсовой работы были успешно реализованы две программы на ассемблере RISC-V. Первая программа преобразует целое число в его текстовое представление, демонстрируя работу с макросами, системными вызовами и обработкой данных. Вторая программа реализует простейший планировщик задач с циклическим переключением, имитируя базовую многозадачность.

Обе задачи позволили углубить понимание архитектуры RISC-V, принципов работы с памятью, системными прерываниями и организацией программного кода. Полученные навыки могут быть применены в дальнейшем для разработки более сложных системных приложений.

Список литературы

1. RISC-V с нуля // Хабр URL: <https://habr.com/ru/articles/454208/> (дата обращения: 14.11.2025).
2. Изучаем RISC-V с нуля, часть 1: Ассемблер и соглашения // Хабр URL: <https://habr.com/ru/articles/533272/> (дата обращения: 14.11.2025).
3. Изучаем RISC-V с нуля, часть 2: прерывания и стыковка с Си // Хабр URL: <https://habr.com/ru/articles/533356/> (дата обращения: 14.11.2025).